

Práctica: "Analizador de Fitness Inteligente" (Contador de Repeticiones)

Objetivo: Crear un programa que no solo detecte la pose, sino que utilice la **geometría** para contar repeticiones de un ejercicio (por ejemplo, sentadillas o flexiones).

- **Lenguaje:** Python.
- **Librerías:** `ultralalytics`, `opencv-python`.
- **Elemento:** Un video de alguien haciendo ejercicio.

Otras aplicaciones posibles:

- Detector de mala postura (avisar si el cuello está muy inclinado hacia adelante).
- Monitor de Yoga (verificar si los brazos están a 180°).

2. Estructura de la práctica

1. **Detección básica:** Cargar el modelo `yolo26n-pose.pt` y mostrar los puntos clave en pantalla.
2. **Cálculo de ángulos:** Identificar los índices de los puntos (ej. Cadera: 11/12, Rodilla: 13/14, Tobillo: 15/16) y calcular el ángulo de la articulación usando trigonometría (arcotangente).
3. **Lógica de conteo:**
 - Si el ángulo es $> 160^\circ$: Estado "Arriba".
 - Si el ángulo es $< 90^\circ$: Estado "Abajo".
 - Contar una repetición cuando pase de "Abajo" a "Arriba".

Usa el modelo preentrenado

Puedes encontrar información oficial técnica en: <https://docs.ultralalytics.com/es/tasks/pose/>

`yolo26n-pose.pt` ya sabe detectar los 17 puntos del cuerpo humano perfectamente. Enfócate en la lógica de programación (qué hacer con esos puntos).

Ejemplo sencillo de código para comenzar

```
import cv2
from ultralytics import YOLO

# 1. Cargar el modelo YOLO26-pose
model = YOLO("yolo26n-pose.pt")

# 2. Ruta del archivo
path_al_video = "ejercicio_fitness.mp4"
cap = cv2.VideoCapture(path_al_video)

# Alternativa con webcam

# --- Configuración ---
model = YOLO("yolo26n-pose.pt")
cap = cv2.VideoCapture(0) # 0 para webcam
```

Geometría y cálculo de ángulos

Para calcular ángulos entre articulaciones en estimación de pose, utilizamos **geometría vectorial**. Necesitamos siempre tres puntos (coordenadas x,y) para definir un ángulo.

Por ejemplo, para el **ángulo del codo**, necesitamos:

1. **A (Hombro):**
(x1,y1)(x1,y1)
2. **B (Codo) (Vértice del ángulo)**
(x2,y2)(x2,y2)
3. **C (Muñeca):**
(x3,y3)(x3,y3)

Aquí tienes la **implementación matemática**

La Fórmula Matemática (Arco Tangente)

La forma más robusta de calcular el ángulo en un plano 2D es usando la función

$\text{atan2}(y,x)$, que devuelve el ángulo en radianes respecto al eje X.

Paso 1: Calcular los ángulos de los dos vectores (Vértice a Punto A y Vértice a Punto C):

$$\theta_1 = \text{atan2}(y_1 - y_2, x_1 - x_2) \quad \theta_1 = \text{atan2}(y_1 - y_2, x_1 - x_2)$$

$$\theta_2 = \text{atan2}(y_3 - y_2, x_3 - x_2) \quad \theta_2 = \text{atan2}(y_3 - y_2, x_3 - x_2)$$

Paso 2: Calcular la diferencia y convertir a grados:

$$A'ngulo = |\theta_2 - \theta_1| \cdot (180/\pi) \quad A'ngulo = |\theta_2 - \theta_1| \cdot (\pi/180)$$

Paso 3: Normalización (Opcional):

Si el ángulo resultante es mayor a 180°, a veces se prefiere el ángulo interior:

$$\text{Si } a'ngulo > 180: a'ngulo = 360 - a'ngulo \quad \text{Si } a'ngulo > 180: a'ngulo = 360 - a'ngulo$$

Implementación en Python

En programación, no usamos la Ley del Coseno porque es computacionalmente más lenta y propensa a errores de precisión. Usamos `math.atan2`.

Podéis integrar directamente esta función:

```
import math

def calcular_angulo(A, B, C):
    """
    Calcula el ángulo en el punto B.
    A, B, C son listas o tuplas [x, y]
    """
    # 1. Obtener radianes con atan2(y, x)
    # Restamos las coordenadas del vértice (B) para centrar el vector
    # en el origen
    radianes = math.atan2(C[1] - B[1], C[0] - B[0]) - \
        math.atan2(A[1] - B[1], A[0] - B[0])

    # 2. Convertir a grados y valor absoluto
    angulo = abs(radianes * 180.0 / math.pi)

    # 3. Asegurar que el ángulo esté entre 0 y 180
    if angulo > 180.0:
```

```

    angulo = 360 - angulo

    return angulo

# Ejemplo de uso (basado en los índices de YOLO):
# hombro = [keypoints[5][0], keypoints[5][1]]
# codo = [keypoints[7][0], keypoints[7][1]]
# muneca = [keypoints[9][0], keypoints[9][1]]
# print(calcular_angulo(hombro, codo, muneca))

```

3. Guía de puntos clave para los alumnos

Para que la práctica funcione, dales esta tabla de referencia de los índices de YOLO:

Articulación	Vértice (B)	Punto A	Punto C
Codo Izquierdo	7 (Codo)	5 (Hombro)	9 (Muñeca)
Codo Derecho	8 (Codo)	6 (Hombro)	10 (Muñeca)
Rodilla Izq.	13 (Rodilla)	11 (Cadera)	15 (Tobillo)
Rodilla Der.	14 (Rodilla)	12 (Cadera)	16 (Tobillo)
Hombro (Axila)	5 (Hombro)	7 (Codo)	11 (Cadera)

Reto adicional:

Dibujar el ángulo con `cv2.putText` justo al lado del vértice (punto B). Y de esta manera ver en tiempo real cómo cambia el número mientras se mueven.

1. Cómo extraer las coordenadas (Paso a Paso)

Cuando ejecutas `results = model(frame)`, obtienes una lista. Para el primer humano detectado, las coordenadas están en `results[0].keypoints.xy`.

Importante: Las coordenadas vienen en un "Tensor" de PyTorch, extraídas con `results[0].keypoints.xy` (el tensor).

Es importante mirar su "**Shape**" (**forma**).

Si el modelo detecta a una persona, el tensor tendrá una forma de **(1, 17, 2)**:

- **1**: El número de personas detectadas.
- **17**: Los puntos clave (Nariz, ojos, hombros, codos...).
- **2**: Las coordenadas **X** e **Y**.

Ejemplo:

Es como una tabla de 17 filas donde la columna 1 es la X y la columna 2 es la Y.

Hay que convertirlas a números enteros para que OpenCV pueda dibujar con ellas.

```
# Suponiendo que 'results' es la salida del modelo
for result in results:
    # .xy[0] nos da los 17 puntos del PRIMER humano detectado
    # La forma es (17, 2) -> 17 filas, cada una con [x, y]
    kpts = result.keypoints.xy[0]

    if kpts.shape[0] > 0: # Verificar que se detectó a alguien
        # Ejemplo: Extraer el Codo Izquierdo (Índice 7)
        # .item() convierte el tensor a un número normal de Python
        x_codo = int(kpts[7][0].item())
        y_codo = int(kpts[7][1].item())

        punto_codo = (x_codo, y_codo)
```

2. Dibujar el ángulo en el vértice

Para que se vea profesional, no solo pondremos el número, sino que dibujaremos líneas que unan los puntos.

Crea un programa que:

1. Detecte la pose.
2. Extrae Hombro, Codo y Muñeca (lado izquierdo).
3. Calcula el ángulo.
4. **Dibuja líneas** entre las articulaciones.

5. **Escribe el ángulo** justo encima del codo.

INICIO DEL PROGRAMA

```
import cv2
from ultralytics import YOLO
import math

# --- Función de apoyo ---
def calcular_angulo(A, B, C):
    radianes = math.atan2(C[1] - B[1], C[0] - B[0]) - \
        math.atan2(A[1] - B[1], A[0] - B[0])
    angulo = abs(radianes * 180.0 / math.pi)
    if angulo > 180.0:
        angulo = 360 - angulo
    return angulo

# --- Configuración ---
model = YOLO("yolo26n-pose.pt")
```

```
// MÉTODO: cv2.VideoCapture("ruta_video")
// Prepara el flujo de datos desde el archivo de video
video = ABRIR_VIDEO("ejercicio.mp4")
```

MIENTRAS el video tenga imágenes disponibles: `while cap.isOpened():`

```
// MÉTODO: video.read()
// Extrae el cuadro (frame) actual para procesarlo
imagen = LEER_SIGUIENTE_CUADRO(video)
SI no hay imagen: ROMPER BUCLE

// MÉTODO: modelo(imagen)
// La IA analiza la imagen y genera una lista de resultados
prediccion = ANALIZAR_CON_IA(modelo, imagen)
```

```

// ACCESO A DATOS: prediccion[0].keypoints.xy

results = model(frame, verbose=False)

for r in results:
    kpts = r.keypoints.xy[0] # Puntos del primer humano

// Se extrae el Tensor que contiene las 17 articulaciones (X, Y)
lista_puntos = EXTRAER_TENSORES_DE_COORDENADAS(prediccion)

Si la lista_puntos no está vacía:

// Seleccionamos los índices específicos de YOLO
punto_A = lista_puntos[5] // Hombro Izquierdo

    hombro = (int(kpts[5][0]), int(kpts[5][1]))

punto_B = lista_puntos[7] // Codo Izquierdo (Vértice)
punto_C = lista_puntos[9] // Muñeca Izquierda

// PROCESO: Conversión de datos
// Pasamos de valores de Tensor (PyTorch) a Números Enteros (Python/OpenCV)
coordenada_A = CONVERTIR_A_ENTERO(punto_A)
coordenada_B = CONVERTIR_A_ENTERO(punto_B)
coordenada_C = CONVERTIR_A_ENTERO(punto_C)

// FUNCIÓN MATEMÁTICA: math.atan2()
// Calculamos la apertura entre los vectores BA y BC

    ang = calcular_angulo(hombro, codo, muneca)

// MÉTODO: cv2.line() y cv2.putText()

// Modificamos la imagen original para añadir información visual
DIBUJAR_LINEA(imagen, entre: coordenada_A y coordenada_B)

    # 2. Dibujar líneas que conectan los puntos (Brazo)
    cv2.line(frame, hombro, codo, (255, 255, 255), 3) #

```

Blanco

DIBUJAR_LINEA(imagen, entre: coordenada_B y coordenada_C)

ESCRIBIR_TEXTO(imagen, "Ángulo: " + angulo, en la posición de coordenada_B)

```
# 3. Dibujar círculos en las articulaciones
cv2.circle(frame, hombro, 5, (0, 0, 255), -1) # Rojo

# 4. ESCRIBIR EL ÁNGULO EN EL VÉRTICE (Codo)
# cv2.putText(imagen, texto, posición, fuente, escala, color,
grosor)
```

FIN SI

// MÉTODO: cv2.imshow()

// Muestra la ventana con el video y los dibujos en tiempo real

MOSTRAR_VENTANA("Resultado", imagen)

// MÉTODO: cv2.waitKey()

// Pequeña pausa para que el ojo humano vea el video (ej. 30ms)

SI el usuario presiona la tecla 'q': ROMPER BUCLE

FIN MIENTRAS

// MÉTODO: video.release()

// Libera el archivo de video y cierra las ventanas

CERRAR_TODO()

FIN DEL PROGRAMA

Trucos:

1. **Filtro de confianza:** A veces YOLO detecta puntos erróneos si la persona está muy lejos. Pueden usar `results[0].keypoints.conf` para solo dibujar el ángulo si la confianza de los 3 puntos es mayor a 0.5.
2. **Color dinámico:** Reto interesante:
 - *Si el ángulo está entre 160 y 180 (brazo estirado), el texto es Verde.*
 - *Si el ángulo es menor a 90 (brazo flexionado), el texto es Rojo.*
3. **Uso de `int(kpts[idx][0])`:** OpenCV no acepta números flotantes (decimales) para las coordenadas de los píxeles; siempre hay que redondear o convertir a `int`.

Si es para sentadillas, cambiar los índices a **11 (Cadera)**, **13 (Rodilla)** y **15 (Tobillo)**. La lógica es exactamente la misma.